

CS6360 Assignment 2

Mathematical (Application) / Theoretical Concepts

**Compositional Learning of Dynamical System Models Using Port
Hamiltonian Neural Networks**

Ghanendra Singh, AI, IITH

Mathematical Concepts

- Port Hamiltonian Systems Theory
- Dirac Structures (dual spaces - Theory)
- Adjoint Sensitivity (Numerical Optimization - Application NN)

Mathematical Concepts

Port Hamiltonian Systems Theory (PHS)

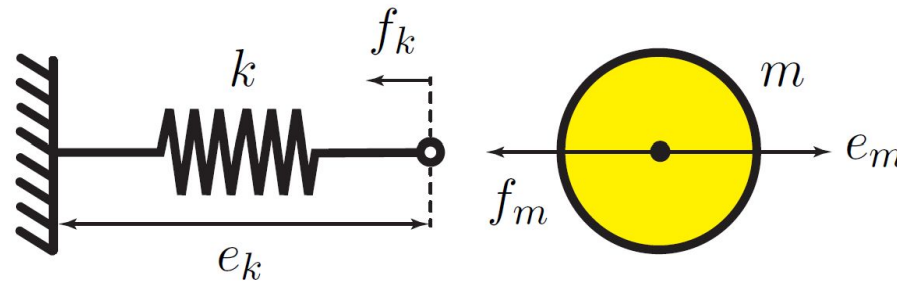
To provide a unified framework for the modeling of systems belonging to different physical domains (mechanical, electrical, thermal, etc.) by

- Energy principles as the common (bridging language).
- The structure of the physical system as a collection of components linked by edges capturing the energy-flows between them.
- Inclusion of energy-dissipating elements, which are largely absent in classical Hamiltonian systems.

Mass Spring System

Consider a point mass with mass m , without friction, under the influence of a spring force corresponding to a linear spring with spring constant k .

Second-order differential equation where z_0 is the rest length.

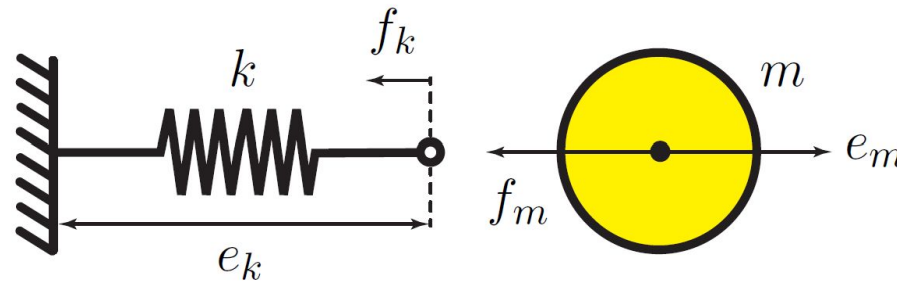


$$m\ddot{z} = -k(z - z_0)$$

Port-based Network Modelling

- A slightly different but equivalent approach *Paynter (1960)*.
- Consider the system as the interconnection of two subsystems, both of which store energy, namely the spring system storing potential energy and the mass system storing kinetic energy.

$$\text{spring: } \begin{cases} \dot{q} = -f_k, \\ e_k = \frac{d}{dq} \left(\frac{1}{2} k q^2 \right) \end{cases}$$



$$\text{mass: } \begin{cases} \dot{p} = -f_m, \\ e_m = \frac{d}{dp} \left(\frac{1}{2m} p^2 \right) \end{cases}$$

Hamiltonian equations

$$\begin{bmatrix} \dot{q} \\ \dot{p} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} \frac{\partial H}{\partial q}(q, p) \\ \frac{\partial H}{\partial p}(q, p) \end{bmatrix}, \quad H(q, p) = \frac{1}{2} k q^2 + \frac{1}{2m} p^2$$

$$\text{interconnection: } \begin{cases} -f_k = e_m, \\ f_m = e_k, \end{cases}$$

q : elongation of the spring
 f_k : velocity of endpoint of spring
 $e_k = kq$: spring force
 f_m : force exerted on the mass
 e_m : velocity of the mass

Port Hamiltonian Systems

Extends to general physical systems, by replacing the earlier power conserving interconnection element by the geometric notion of a Dirac structure

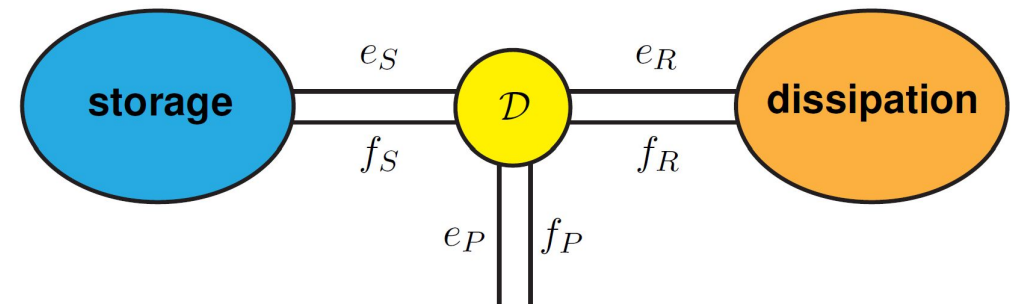
In port based modelling, large scale systems consists of Interconnection of three types of components. Linked via pairs (f, e) of equally dimensioned vectors of **flow** and **effort** variables.

- energy-storing elements: **S (storage)**
- energy-dissipating elements: **R (resistive)**
- energy-routing elements. **D (Dirac)**

Three ports

- port (f_S, e_S) linking to energy storage
- port (f_R, e_R) corresponding to energy-dissipation, and
- external port (f_P, e_P) , by which the system interacts with its environment (including regulation)

Ref: Arjan van der Schaft – PHS Theory



Products denote energy transmission: $e_S^T f_S, e_R^T f_R, e_P^T f_P$

Dirac Structure

Think as a '*printed circuit board*' which provides the wiring for the system.

Basic property of a Dirac structure is *power conservation*.

Relate power(energy) variables of composing elements of the system in a power conserving manner.

Power(energy) variables appear in conjugate pairs. (VI, mv^2, kx^2)

Mathematically they are modeled to take their values in *dual linear spaces*.

Link various port (flow and effort) variables so that the total power $\mathbf{e}^T \mathbf{f} = \mathbf{0}$.

Composition of Dirac structures again defines a Dirac structure (D).

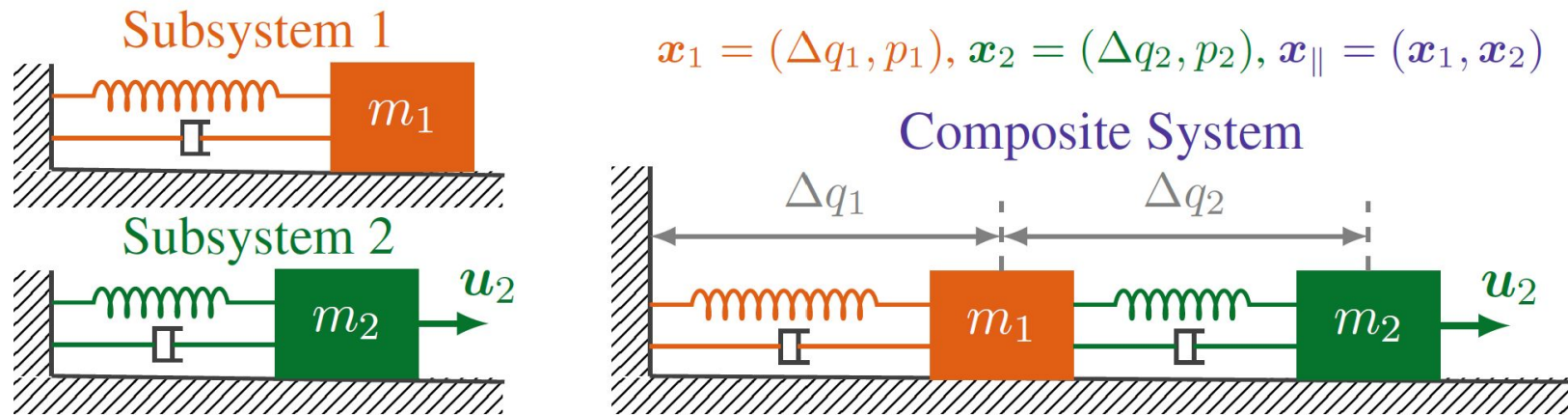
Port-Hamiltonian Systems Model

where $\dot{x} = [J(x) - R(x)] \nabla_x H(x) + G(x)u, \quad y = G(x)^T \nabla_x H(x)$

- $x \in R^n$ is n dimensional vector representing the system's state
- $H(x)$ is the system's Hamiltonian function
- $J(x) \in R^{n \times n}$ is the skew symmetric interconnection matrix (i.e. $J(x) = -J(x)^T$ for every $x \in R^n$)
- $R(x) \in R^{n \times n}$ is the symmetric positive semi definite dissipation matrix
- $G(x) \in R^{n \times m}$ is control input matrix
- $u \in R^m$ is m dimensional control input vector and $y \in R^m$ is output vector

$H(x)$ describes the system's energy in terms of its state x , $J(x)$ encodes the energy-conserving interactions between the various components of the system and $R(x)$ encodes how these components dissipate energy.

Composite System



$$H_i(\mathbf{x}_i) = \frac{p_i^2}{2m_i} + \frac{k\Delta q_i^2}{2}$$

Consider a composite system as interconnection of two **mass-spring** subsystems, both of which store energy, namely the spring system storing potential energy and the mass system storing kinetic energy.

$$\mathbf{p}H_1(\mathbf{x}_1) + \mathbf{p}H_2(\mathbf{x}_2) = \mathbf{p}H_c(\mathbf{x}_c)$$

Composite System Dynamics

Dynamics of the individual spring-mass-damper systems are given by equation:

$$\dot{x} = [J(x) - R(x)]\nabla_x H(x) + G(x)u, \quad y = G(x)^T \nabla_x H(x)$$

$$J_i = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad R_i(x_i) = \begin{bmatrix} 0 & 0 \\ 0 & b_i \frac{p_i^2}{m_i^2} \end{bmatrix}, \quad G_i = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Hamiltonian of the composite function: $H_c(x_c) = H_1(x_1) + H_2(x_2)$

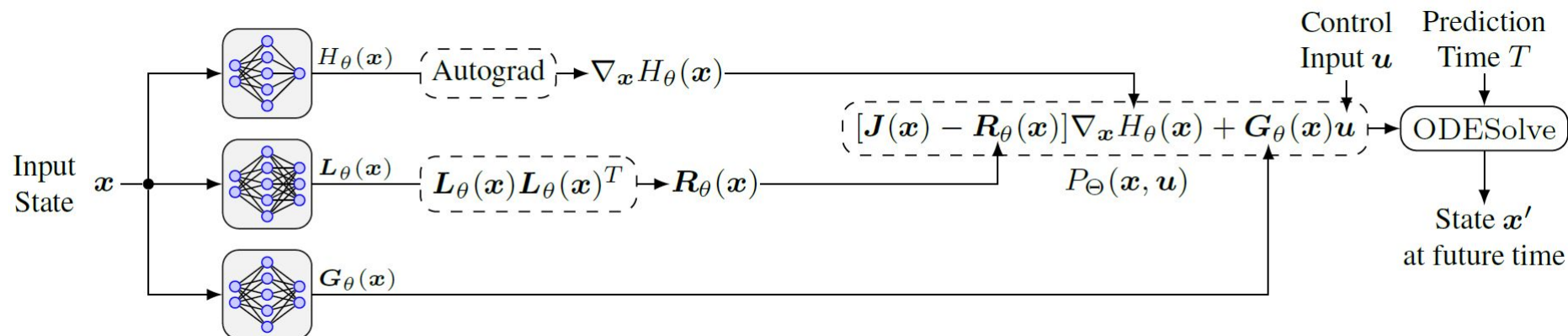
$$J_c = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix}, \quad R_c(x_c) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & b_1 \frac{p_1^2}{m_1^2} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & b_2 \frac{p_2^2}{m_2^2} \end{bmatrix}, \quad G_c = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Composite Dynamics

$$\dot{x}_c = [J_c(x_c) - R_c(x_c)]\nabla_x H(x_c) + G(x_c)u, \quad y = G(x_c)^T \nabla_x H(x_c)$$

Constructing PHNN

- Construction and evaluation of PHNN.
- $H(x)$, $R(x)$, and $G(x)$, denote parametrizations terms in $P_{\Theta}(x, u)$
- $H(x)$ is parametrized as a multi-layer perceptron (MLP).
- $R_{\theta(x)} = L_{\theta}(x)L_{\theta}(x)^T$ (parameterized Lower triangular matrix).
- $J(x)$ interaction term is known priori.



Training PHNN

- Assume that a finite dataset $\mathcal{D}$ of system trajectories time-series data of states and control inputs
- A set $\mathcal{D} = \{\tau_1, \dots, \tau_{|\mathcal{D}|}\}$ of trajectories $\tau = \{(x_0, u_0, t_0), \dots, (x_{|\tau|}, u_{|\tau|}, t_{|\tau|})\}$
- x_i is the state at time t_i , u_i is the control applied at time t_i
- Loss function of the PHNN to capture error in model's prediction

$$\mathcal{L}(\Theta, \mathcal{D}) = \sum_{\tau_l \in \mathcal{D}} \sum_{(x_i, u_i, t_i) \in \tau_l} \|PHNN_{\Theta}(x_i, u_i, t_i, t_{i+1}) - x_{i+1}\|^2$$

- Local minima of the loss $\mathcal{L}(\Theta, \mathcal{D})$ using gradient based techniques.
- $\nabla_{\Theta} \mathcal{L}(\Theta, \mathcal{D})$ is computed using automatic differentiation **ODE Solve** or using ***adjoint sensitivity method***.

Adjoint Sensitivity Method

- Computes gradients by solving a 2nd order ODE reverse in time.
- Scales linearly with parameters and state.
- Efficient to calculate large systems gradient.
- System contains both the original state and the sensitivity of the loss with respect to the state.
- If Loss depends directly on multiple states , update adjoint state in the direction $\mathbf{a}(t)$
- Adjoint $\mathbf{a}(t) = \frac{\partial L}{\partial \mathbf{z}}(t)$: Finding loss w.r.t $\mathbf{z}(t)$.
- Optimizing a scalar-valued loss function with input from ODE Solver:

$$L(\mathbf{z}(t_1)) = L \left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta) dt \right) = L(\text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_1, \theta))$$

Ref: Neural ODEs (Chen et al. 2018)

